

Lightweight NSFW detection models for mobile mental health apps

****For on-device Android content filtering in addiction recovery apps, NSFWJS with MobileNetV2 emerges as the optimal solution****—it's the only model under 5MB that provides multi-class detection including a dedicated "Sexy" category for suggestive content. NudeNet offers complementary granularity by detecting covered body parts. Both achieve real-time inference (15-40ms) on mid-range devices and can be self-hosted on CPU-only servers. Binary classifiers like Falconsai achieve higher headline accuracy (98%+) but cannot distinguish triggers that fall short of explicit nudity—a critical limitation for recovery applications prioritizing recall.

The landscape of open-source NSFW models

The detection model ecosystem divides into two camps: ****binary classifiers**** (NSFW vs. safe) and ****multi-class models**** that separate explicit content from suggestive material. For mental health trigger suppression, this distinction is decisive.

****NSFWJS**** (GitHub: infinitered/nsfwjs) provides the most deployment-ready solution for your use case. Built on transfer learning from MobileNetV2, it classifies images into five categories: ****Neutral, Drawing, Hentai, Porn, and Sexy****. [github](https://github.com/GantMan/nsfw_model) [GitHub] (https://github.com/infinitered/nsfwjs) The "Sexy" class specifically captures revealing clothing, suggestive poses, and intimacy without nudity—exactly the borderline content that binary models miss. [InfoQ](https://www.infoq.com/news/2019/04/nsfw-machine-learning-rating/) Model size ranges from ****~4MB**** (MobileNetV2 small) to 22MB (mid) to 87MB (full InceptionV3), with accuracy scaling from ****~90% to ~93%**** [GitHub](https://github.com/infinitered/nsfwjs) [InfoQ] (https://www.infoq.com/news/2019/04/nsfw-machine-learning-rating/) respectively. The MIT-licensed TensorFlow.js implementation runs natively in browsers [github](https://github.com/infinitered/nsfwjs) and converts readily to TFLite.

****NudeNet v3**** takes an object detection approach using YOLOv8, returning bounding boxes for 18 body part categories. Critically, it detects both exposed ***and*** covered anatomy—`FEMALE\_BREAST\_COVERED`, `FEMALE\_GENITALIA\_COVERED` — [GitHub](https://github.com/vladmandic/nudenet) enabling detection of suggestive clothing without explicit nudity. [GitHub](https://github.com/w-e-w/sd-webui-nudenet-nsfw-censor) The nano model (`320n`) weighs just ****~6MB**** in ONNX format. Licensed under AGPLv3, it requires open-sourcing derivative works, which may complicate commercial app deployment.

Model	Architecture	Size	Classes	Suggestive Detection	License
NSFWJS (small)	MobileNetV2	~4MB	5	✓ "Sexy" class	MIT
NudeNet 320n	YOLOv8n	~6MB	18	✓ Covered parts	AGPLv3
Marqo/nsfw-384	ViT Tiny	~20MB	2	✗ Binary only	Apache-2.0
Falconsai	ViT Base	~343MB	2	✗ Binary only	Apache-2.0

Yahoo OpenNSFW, the pioneering model from 2016, uses a thin ResNet-50 variant [Wolfram] (<https://resources.wolframcloud.com/NeuralNetRepository/resources/Yahoo-Open-NSFW-Model-V1/>) (~24MB) but is archived and binary-only. Its modern Python wrapper **OpenNSFW2** (pip-installable, MIT license) remains maintained [GitHub] (<https://github.com/bhky/opennsfw2>) [PyPI] (<https://pypi.org/project/opennsfw2/>) but inherits the binary limitation.

Multi-class models on Hugging Face fill the granularity gap

For distinguishing explicit from suggestive content, several Hugging Face models warrant consideration:

LukeJacob2023/nsfw-image-detector replicates NSFWJS's five-class taxonomy (Drawing, Hentai, Neutral, Porn, Sexy) using a ViT backbone, achieving **93.16% accuracy** with 98.87% top-k accuracy. [Hugging Face] (<https://huggingface.co/LukeJacob2023/nsfw-image-detector>) At 86M parameters, it requires ONNX conversion for mobile deployment but provides the granularity needed for trigger filtering.

prithivMLmods/siglip2-x256-explicit-content offers a semantically clearer taxonomy with an explicit **"Enticing or Sensual"** category alongside Pornography, Hentai, Anime, and Normal. Built on SigLIP2, it achieves **91.37% overall accuracy** with **96.54% recall on pornography**— [huggingface] (<https://huggingface.co/prithivMLmods/siglip2-x256-explicit-content>) strong performance for high-recall requirements. Its 93M parameters and 256×256 input make it convertible to TFLite but heavier than MobileNet alternatives.

MichalMlodawski/nsfw-image-detection-large takes a three-tier approach: Safe, **Questionable**, Unsafe. The "Questionable" category maps directly to borderline content, though the FocalNet architecture and 512×512 input increase compute requirements. [Aibase] (<https://model.aibase.com/models/details/1915694785098309633>) [Hugging Face] (<https://huggingface.co/TostAI/nsfw-image-detection-large>)

The most downloaded model, **Falconsai/nsfw_image_detection** (80M+ downloads, 98% accuracy), uses ViT-base but provides only binary classification. [Hugging Face] (https://huggingface.co/Falconsai/nsfw_image_detection) It **cannot distinguish a swimsuit photo from hardcore pornography**—both return "nsfw." For recovery apps needing nuanced filtering, this is disqualifying despite the headline accuracy.

Android deployment achieves real-time inference at minimal footprint

On-device NSFW detection on Android is production-ready with **15-40ms inference times** using models under 5MB. TensorFlow Lite provides the most mature ecosystem with the best performance on Android.

Model architecture choices directly impact mobile viability:

- **MobileNetV2**: ~14MB (FP32) → **~3.5MB** (INT8 quantized), [viraanhub] (<https://viraanhub.com/machine-learning-model-optimization-for-edge-devices-complete-guide>) 20-40ms on mid-range Android
- **EfficientNet-Lite0**: ~17MB (FP32) → **~4.4MB** (INT8), 15-25ms on Pixel 4
- **ViT models**: 86-93M parameters (~344MB), require aggressive quantization for mobile

Quantization reduces size 4× with minimal accuracy loss. INT8 quantization using TFLite's converter with a representative calibration dataset typically costs 1-3% accuracy while enabling hardware acceleration via Android's NNAPI. Float16 quantization (2× reduction, negligible accuracy loss) works well with GPU delegates.

For rapid implementation, **nsfw-detector-android** (GitHub: nipunru/nsfw-detector-android) provides a production-ready Kotlin library using Firebase ML Kit with TFLite: [github] (<https://github.com/nipunru/nsfw-detector-android>)

```
```kotlin
NSFWDetector.isNSFW(bitmap, 0.7f) { isNSFW, confidence ->
 // Filter content based on result
}
```

**Framework benchmarks on Samsung S21** show TFLite outperforming alternatives for image classification:

- TensorFlow Lite: ~23ms, ~89MB memory
- ONNX Runtime: ~31ms, ~112MB memory
- PyTorch Mobile: ~38ms, ~126MB memory [DigitalOcean] (<https://www.digitalocean.com/community/tutorials/ai-model-deployment-optimization>)

ONNX Runtime offers better cross-platform flexibility if you need identical models on iOS/Android/server, but TFLite wins on pure Android performance. [Dev-kit](<https://dev-kit.io/blog/machine-learning/onnx-vs-pytorch-speed-comparison>)

**## Server deployment runs efficiently on CPU-only infrastructure**

For self-hosted fallback or batch processing, containerized solutions enable deployment without GPU dependencies.

**Docker quick-start options:**

**tmplink/nsfw\_detector** wraps Falconsai's ViT model with multi-CPU utilization, video/PDF support, and configurable thresholds. Requires ~2GB memory under load: [github] ([https://github.com/tmplink/nsfw\\_detector](https://github.com/tmplink/nsfw_detector)) [GitHub]([https://github.com/tmplink/nsfw\\_detector](https://github.com/tmplink/nsfw_detector))

```
```bash
docker run -d -p 3333:3333 vxlink/nsfw_detector:latest
```

```
curl -X POST -F "file=@image.jpg" http://localhost:3333/check
```

...

****NudeNet's official container**** provides object-level detection:

```
```bash
```

```
docker run -it -p8080:8080 ghcr.io/notai-tech/nudenet:latest
```

...

[PyPI](https://pypi.org/project/nudenet/)

**\*\*ONNX Runtime optimization\*\*** dramatically improves CPU inference. Key configurations: set

``intra_op_num_threads=1`` for containerized deployments with Kubernetes HPA, disable spinning

(``session.intra_op.allow_spinning=False``) to reduce CPU usage from 47% to 0.5% between requests,

[Inworld AI](https://inworld.ai/blog/reducing-cpu-usage-in-machine-learning-model-inference-with-onnx-runtime) and apply INT8 quantization for 2-4× model size reduction.

**\*\*For high-volume deployments\*\***, Triton Inference Server's dynamic batching improves throughput 2-4×— benchmarks show Inception models jumping from 73 infer/sec to 267 infer/sec with batch sizes of 4-8.

[NVIDIA](https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user\_guide/optimization.html)

Academic benchmarks (arXiv:2312.16338) on the LSPD dataset show **\*\*CNNs outperforming Vision Transformers\*\*** for NSFW classification: ConvNexT (tiny) achieves 0.952 F1 versus ViT (B16) at 0.920.

MobileNetV3 (large) hits 0.949 F1— [arxiv](https://arxiv.org/html/2312.16338v1) [arXiv]

(https://arxiv.org/html/2312.16338v1) strong evidence that lightweight CNN architectures don't sacrifice accuracy for this task.

## Detection accuracy on borderline content remains the critical limitation

Developer community experiences reveal systematic weaknesses in suggestive content detection that directly impact your use case:

**\*\*Binary models consistently fail on borderline content.\*\*** Swimwear, shirtless photos, artistic poses, and revealing-but-clothed images fall into an ambiguous zone where models trained primarily on pornography struggle. [Bomberbot](https://www.bomberbot.com/machine-learning/building-robust-nsfw-detectors-with-deep-learning-a-comprehensive-guide/) The fundamental problem: training datasets typically contain explicit pornography and clearly safe images, with sparse representation of the suggestive middle ground.

**\*\*Documented bias patterns\*\*** affect recovery applications: women flagged at **\*\*17.9% higher rates\*\*** than men; [Hugging Face](https://huggingface.co/CompVis/stable-diffusion-safety-checker) eating/open-mouth images frequently misclassified; young adults (15-29) show highest false positive rates. [ACM Digital Library](https://dl.acm.org/doi/fullHtml/10.1145/3630106.3658963) Models may function as "human detectors" rather than "nudity detectors" if training data lacks sufficient clothed humans. [Medium]

(<https://praneethbedapudi.medium.com/nudenet-an-ensemble-of-neural-nets-for-nudity-detection-and-censoring-d9f3da721e3>)

**\*\*Multi-class models partially address this\*\*** through explicit "Sexy/Sensual" categories, but accuracy on this intermediate class runs 5-10% lower than on explicit content. NSFWJS documentation acknowledges challenges with the "Sexy" category boundary.

**\*\*Ensemble approaches improve coverage.\*\*** Running multiple models—e.g., NSFWJS for classification plus NudeNet for body part detection—catches images that slip past individual models. The computational overhead is acceptable on-device given sub-40ms inference per model.

## ## Practical recommendations for mental health content filtering

**\*\*For your primary use case\*\*** (Android on-device, prioritizing recall for trigger suppression), the recommended stack:

**\*\*1. Primary model: NSFWJS MobileNetV2 (~4MB TFLite)\*\***

- Convert the MobileNetV2 variant to TFLite with INT8 quantization
- The "Sexy" class provides explicit handling of suggestive content
- Set low classification thresholds (0.3-0.5) for high recall
- Expected inference: 20-40ms on mid-range devices

**\*\*2. Secondary detection: NudeNet 320n (~6MB ONNX)\*\***

- Complements classification with body part detection
- "Covered" categories catch revealing clothing
- Run when NSFWJS returns ambiguous scores
- Note AGPLv3 license implications for commercial deployment

**\*\*3. Threshold strategy for recall over precision:\*\***

- Flag content when Porn + Hentai + Sexy probabilities sum > 0.4
- Accept higher false positives as acceptable cost for blocking triggers
- Provide user controls to adjust sensitivity

**\*\*For server fallback\*\***, deploy [tmplink/nsfw\\_detector](https://github.com/Falconsai/vit-base-nsfw-detector) (Falconsai ViT) for simplicity or NudeNet for granularity. Both run on 2-core CPU instances with ~2GB RAM.

**\*\*Known edge cases to handle:\*\***

- Artistic/museum content (consider allowlisting specific apps/domains)
- Medical imagery (may require user override)
- Illustrated/anime content (NSFWJS handles via Drawing/Hentai classes)
- AI-generated images (some models show 10%+ accuracy drop on synthetic content) [Hugging Face] (<https://huggingface.co/AdamCodd/vit-base-nsfw-detector>) [huggingface] (<https://huggingface.co/AdamCodd/vit-base-nsfw-detector>)

## ## Conclusion

The optimal architecture combines \*\*NSFWJS's multi-class MobileNetV2\*\* for primary detection with \*\*NudeNet's body part localization\*\* for ambiguous cases. Both achieve mobile-viable footprints under 6MB with real-time inference. The critical insight: headline accuracy metrics on binary NSFW/SFW classification obscure poor performance on the suggestive content most relevant to recovery applications. Multi-class models with explicit "Sexy/Sensual" categories—despite lower overall accuracy—better serve high-recall use cases. For server deployment, containerized solutions run comfortably on CPU-only infrastructure with ONNX Runtime optimization. The AGPLv3 license on NudeNet requires evaluation against your commercial plans; NSFWJS's MIT license presents no such restrictions.